Example 4-10. Counting Sort implementation

```
/** Sort the n elements in ar, drawn from the values [0,k). */
int countingSort (int *ar, int n, int k) {
  int i, idx = 0;
  int *B = calloc (k, sizeof (int));

  for (i = 0; i < n; i++) {
    B[ar[i]]++;
  }

  for (i = 0; i < k; i++) {
    while (B[i]-- > 0) {
       ar[idx++] = i;
    }
  }

  free(B);
}
```


Analysis

COUNTING SORT makes two passes over the entire array. The first processes each of the n elements in the input array. In the second pass, the inner `while` loop is executed $B[i]$ times for each of the $0 \le i < k$ buckets; thus the statement `ar[idx++]` executes exactly n times. Together, the key statements in the implementation execute a total of 2^*n times, resulting in a total performance of $O(n)$.

COUNTING SORT can only be used in limited situations because of the constraints that the elements in the array being sorted are drawn from a limited set of k elements. We now discuss BUCKET SORT, which relaxes the constraint that each element to be sorted maps to a single bucket.

Bucket Sort

COUNTING SORT succeeds by constructing a much smaller set of k values in which to count the n elements in the set. Given a set of n elements, BUCKET SORT constructs a set of n buckets into which the elements of the input set are partitioned; BUCKET SORT thus reduces its processing costs at the expense of this extra space. If a hash function, $\texttt{hash}(A_i)$, is provided that uniformly partitions the input set of n elements into these n buckets, then BUCKET SORT as described in Figure 4-18 can sort, in the worst case, in $O(n)$ time. You can use BUCKET SORT if the following two properties hold:

Uniform distribution
: The input data must be uniformly distributed for a given range. Based on this distribution, n buckets are created to evenly partition the input range.

Ordered hash function
: The buckets must be ordered. That is, if $i<j$, then elements inserted into bucket b_i are lexicographically smaller than elements in bucket b_j.